Parallellprogrammering i Go

Abhinav Sasikumar
Palinda 2026
2026-03-20

Presentationer av palinda-1

Ni kommer att få ett nummer:

1. Task 2
2. Task 3
3. Task 4

Ni ska presentera respektive uppgift för er grupp och diskutera om ni gjorde saker annorlunda osv. Ni ska presentera era diskussioner till resten av klassen.

Agenda

- Parallelism / Concurrency
- Channels
- Race conditions
- Mutex
- Deadlocks
- Aktivitet

Concurrency vs. Parallelism

Concurrency är att hantera många saker samtidigt

-
-

Concurrency vs. Parallelism

Concurrency är att hantera många saker samtidigt
Parallelism är att att göra många saker samtidigt.

-
-

Concurrency vs. Parallelism

Concurrency är att hantera många saker samtidigt
Parallelism är att att göra många saker samtidigt.

- **Concurrency:** byta task på samma CPU
-

Concurrency vs. Parallelism

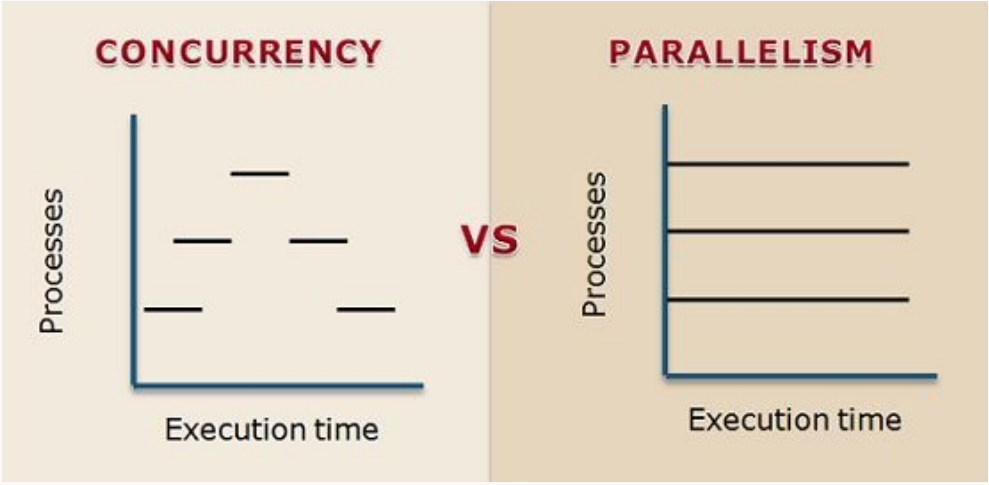
Concurrency är att hantera många saker samtidigt
Parallelism är att att göra många saker samtidigt.

- **Concurrency:** byta task på samma CPU
- **Parallelism:** köra olika tasks på olika CPUer samtidigt.

Concurrency vs. Parallelism

Concurrency är att hantera många saker samtidigt
Parallelism är att att göra många saker samtidigt.

- **Concurrency:** byta task på samma CPU
- **Parallelism:** köra olika tasks på olika CPUer samtidigt.



Goroutines

I Go använder vi inte trådar direkt, utan vi använder **Goroutines**.

-
-

Goroutines

I Go använder vi inte trådar direkt, utan vi använder **Goroutines**.

- Hanteras av Go Runtimen, inte OS.
-

Goroutines

I Go använder vi inte trådar direkt, utan vi använder **Goroutines**.

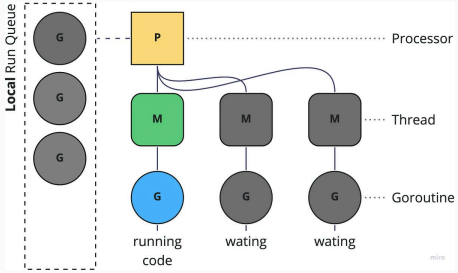
- Hanteras av Go Runtimen, inte OS.
- Multiplexas på OS-trådar automatiskt.

Goroutines

I Go använder vi inte trådar direkt, utan vi använder **Goroutines**.

- Hanteras av Go Runtimen, inte OS.
- Multiplexas på OS-trådar automatiskt.

```go
func main() {
    go doSomething() // Körs nu concurrent!
    fmt.Println("Done.")
}
```



Amdahl's Lag

Speedup man kan få i ett program är begränsat av andelen kod som inte kan paralleliseras

Amdahl's Lag

Speedup man kan få i ett program är begränsat av andelen kod som inte kan paralleliseras

$$\text{speedup} = \frac{1}{(1 - p) + \frac{p}{s}}$$

Amdahl's Lag

Speedup man kan få i ett program är begränsat av andelen kod som inte kan paralleliseras

$$\text{speedup} = \frac{1}{(1 - p) + \frac{p}{s}}$$

Där p är den del av programmet som kan paralleliseras, och s är speedup för den delen.

Amdahl's Lag

Speedup man kan få i ett program är begränsat av andelen kod som inte kan paralleliseras

$$\text{speedup} = \frac{1}{(1 - p) + \frac{p}{s}}$$

Där p är den del av programmet som kan paralleliseras, och s är speedup för den delen.

Slutsats: Om 10% av koden är sekventiell, kan man maximalt få 10x speedup oavsett hur många kärnor man har.

Channel (Kanaler)

“Do not communicate by sharing memory; instead, share memory by communicating.”

Channel (Kanaler)

“Do not communicate by sharing memory; instead, share memory by communicating.”

En kanal tillåter kommunikation mellan goroutiner. Hanterar synkronisering automatiskt genom blockering.

Channel (Kanaler)

“Do not communicate by sharing memory; instead, share memory by communicating.”

En kanal tillåter kommunikation mellan goroutiner. Hanterar synkronisering automatiskt genom blockering.

```
orderChan := make(chan string)

// Skicka till kanal, blockerar om kanalen är full eller obuffrad
orderChan ← "Pepperoni Pizza"

// Läsa från kanal
currentOrder := ←orderChan
```

Demo: Vscode

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

1.
2.
3.

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

Det leder till en race condition eftersom operationen inte är atomisk.

En operation är atomisk om den kan utföras omedelbart och inte kan interruptas.

1.
2.
3.

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

Det leder till en race condition eftersom operationen inte är atomisk.

En operation är atomisk om den kan utföras omedelbart och inte kan interruptas.

En icke-atomisk operation:

1. Read: Läser data från minne till CPU Cache.
2.
3.

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

Det leder till en race condition eftersom operationen inte är atomisk.

En operation är atomisk om den kan utföras omedelbart och inte kan interruptas.

En icke-atomisk operation:

1. Read: Läser data från minne till CPU Cache.
2. Modify: Ändrar värdet i det lokala registret.
3.

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

Det leder till en race condition eftersom operationen inte är atomisk.

En operation är atomisk om den kan utföras omedelbart och inte kan interruptas.

En icke-atomisk operation:

1. Read: Läser data från minne till CPU Cache.
2. Modify: Ändrar värdet i det lokala registret.
3. Write: Skriver värdet tillbaka till RAM.

Race condition

Vad händer om counter = counter + 50 körs i två goroutines samtidigt?

Det leder till en race condition eftersom operationen inte är atomisk.

En operation är atomisk om den kan utföras omedelbart och inte kan interruptas.

En icke-atomisk operation:

1. Read: Läser data från minne till CPU Cache.
2. Modify: Ändrar värdet i det lokala registret.
3. Write: Skriver värdet tillbaka till RAM.

Problem: Den ena goroutinen kan skriva över den andras ändring.

Demo: Vscode

Mutexes: Mutual Exclusion

Om vi måste dela minne använder vi en Mutex (ett lås).

Mutexes: Mutual Exclusion

Om vi måste dela minne använder vi en Mutex (ett lås).

```
var mu sync.Mutex
var locked_variable = 50
func openDoor() {
    mu.Lock()         // Lås variabeln
    defer mu.Unlock() // Väntar tills funktionen returnerar för att låsa
upp

    if cheesePortions > 0 {
        cheesePortions--
    }
  }
```

Demo: Vscode

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

1.
2.

3.
4.

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

Uppstår om fyra konditioner sker samtidigt:

1.
2.

3.
4.

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

Uppstår om fyra konditioner sker samtidigt:

1. Mutex används (Mutual Exclusion)
2.

3.
4.

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

Uppstår om fyra konditioner sker samtidigt:

1. Mutex används (Mutual Exclusion)
2. En process har låst en resurs och väntar på att en annan ska bli fri för at fortsättat
3.
4.

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

Uppstår om fyra konditioner sker samtidigt:

1. Mutex används (Mutual Exclusion)
2. En process har låst en resurs och väntar på att en annan ska bli fri för at fortsättat
3. Resursen kan inte tas från andra processen
4.

Deadlocks

En deadlock uppstår om två trådar väntar på varandra oändligt.

Uppstår om fyra konditioner sker samtidigt:

1. Mutex används (Mutual Exclusion)
2. En process har låst en resurs och väntar på att en annan ska bli fri för at fortsättat
3. Resursen kan inte tas från andra processen
4. Varje process har en resurs som en annan process behöver

Parallel-programmering

Oftast används producer-consumer pattern,

Producer: Producerar något exempelvis data som läggs i en kanal

Consumer: Använder det som har producerats

Aktivitet: Uber i Go

Ni ska göra ett program som hanterar taxi-beställningar

- Producer: Skapar beställningar och lägger i en kanal
- Consumer: Goroutines (taxibilar) som utför körningen.
- Utmaning: Hantera delad data (inkomst) utan race conditions!